

Hi-DoTS 알고리즘 워크스루

처음 보는 사람용 · src/hi_em/hi_dots.py — HiDoTS · 2026-05-22

목차

0. 이 시스템이 푸는 문제 — 온라인 DTS	2
1. Hi-DoTS 는 무엇인가 (한 줄)	2
2. 한 turn 의 동작 순서	3
함수 한눈에	3
STEP 1 — δ 신호 계산	3
함수 <code>_delta_prev(s)</code> — 즉각 표면 변화 (짧은 시간 척도)	3
함수 <code>_delta_ctx(s)</code> — 인과 윈도우 맥락 (긴 시간 척도)	4
함수 <code>_delta_eff(s)</code> — 두 척도 합성	4
STEP 2 — 경계 판정 (assign)	4
STEP 3 — graded score + 상태 갱신 (assign)	5
복잡도	6
3. graded score 를 어떻게 쓰나	6
4. 왜 이게 전부인가 — v4.1.x dead-code 감사 요약	6
5. HP default	7
6. 결과 (공식 SuperDialseg metric, $\text{Score}=0.5F1+0.25(1-Pk)+0.25(1-WD)$)	8
7. 한계 / 검증 미해결	8
검증 노트 (2026-05-22, 코드 1:1 대조)	8

목적: Hi-DoTS segmenter 가 한 turn 을 처리할 때 실제로 무슨 함수를 어떤 순서로 호출하는지, SEM/segmentation 을 처음 보는 사람도 따라올 수 있게 풀어 쓴 문서. 코드와 1:1 대조 검증 완료 (문서 끝 검증 노트).

이 문서엔 안 도는 코드가 없다. Hi-DoTS 는 Hi-EM v4.1.x 라인을 dead-code 감사(2026-05-22)한 끝에 실제로 출력에 영향을 주는 부분만 남긴 reduced-form 모델이다. 그래서 “동작 순서”가 곧 “코드 전부”다. 감사 과정·폐기된 SEM2 기계의 정체는 § 4 에 한 문단으로 요약한다.

0. 이 시스템이 푸는 문제 — 온라인 DTS

대화가 turn 단위로 흐른다. 각 turn 발화는 미리 L2-정규화된 문장 임베딩 s_t 로 들어온다. 매 turn 결정할 것:

- 이 발화에서 새 주제(topic segment)가 시작됐는가? (is_boundary)
- 그리고 그게 얼마나 강한 경계인가 (graded_score)

온라인(online)이라는 제약이 핵심이다 — turn t 의 결정은 **과거 발화 $s_1..s_t$ 만으로** 내려야 한다. 미래 turn은 아직 발화되지 않았다. 실시간 회의 요약·스트리밍 음성 대화·프로액티브 어시스턴트 등 실제 배포 환경이 이걸 강제한다. 그래서 양측을 비교하는 회고적(offline) 방법이 아니라, 과거만으로 현재의 놀람을 추정하는 예측적 방법이 필요하다.

두 벡터의 “비슷함”은 **cosine 유사도** $\cos(a, b)$ 로 잰다(같은 방향 1, 무관 0). 따라서

$$\delta = 1 - \cos(a, b)$$

는 “두 발화가 얼마나 **다른가**” = **놀람의 크기(prediction error, PE)**. Hi-DoTS의 알고리즘 전체는 이 δ 를 두 시간 척도에서 계산해 학습된 임계값 δ^* 와 비교하는 것이다.

1. Hi-DoTS는 무엇인가 (한 줄)

Hi-DoTS = Hi-EM v4.1.x 라인이 경험적으로 환원되는 정직한 최소 온라인 segmenter.

Hi-EM v4.1.x (HiEMSegmenterV411 / V413)는 SEM2 스타일의 무거운 기계 — sticky-CRP prior, 학습형 per-topic RNN, f0/restart 분기, per-topic 분산 calibration — 를 그대로 안고 있었다. 그러나 2026-05-22 감사 결과, v4.1.3 default config에서 그 기계 전부가 **출력에 영향을 주지 않음** (inert)이 증명됐다. Bayesian posterior argmax가 정확히·증명 가능하게 **causal 거리 하나에 대한 단일 임계값 판정**으로 환원된다 (근거 § 4).

Hi-DoTS는 그 환원형을 그대로 독립 클래스로 구현한 것이다. 폐기된 SEM2 기계는 archive/legacy_sem_ablation/에 실행 가능한 채로 보존된다 (감사 계보 코드). Hi-DoTS의 출력은 v4.1.3 default와 **byte 단위로 동일** (TIAGE / Dialseg711 / SuperDialseg에서 검증).

2. 한 turn의 동작 순서

seg.assign(s) 한 번 = 한 turn 처리. 호출되는 함수는 4 개뿐이다 — 거리 함수 3 개와 본체 assign. 아래 표가 기능 전부이고, 이어지는 STEP 1~3 이 이 함수들을 실행 순서대로 하나씩 펼친다.

함수 한눈에

함수	역할	입력	반환	호출 시점
_delta_prev	직전 1 발화 대비 놀람 (짧은 척도)	s	δ_{prev} / None	STEP 1
_delta_ctx	인과윈도우 맥락 대비 놀람 (긴 척도)	s	δ_{ctx} / None	STEP 1
_delta_eff	위 둘을 호출 → 두 척도 합성	s	δ_{eff} (float)	STEP 1
assign	경계 판정 + graded score + 상태 갱신	s	(topic_id, is_boundary)	매 turn 진입점

실행 순서:

STEP 1 δ 신호 계산 assign() → _delta_eff() → _delta_prev() + _delta_ctx()

STEP 2 경계 판정 assign(): $\delta_{\text{eff}} \geq \delta^*$

STEP 3 graded score + 상태 갱신 assign()

함수명은 모두 hi_dots.py 의 실제 메서드명이다. 아래 각 함수 블록은 **역할** / **수식** / **반환** / **의미** 를 같은 형식으로 적는다.

STEP 1 — δ 신호 계산

_delta_eff(s) 가 거리 함수 둘을 호출해 두 시간 척도의 놀람을 계산·합성한다. 함수 3 개를 호출 순서대로 본다.

함수 _delta_prev(s) — 즉각 표면 변화 (짧은 시간 척도)

hi_dots.py [L134](#)

- **역할** — 직전 한 발화와 지금 발화의 거리. 가장 짧은 척도의 놀람.
- **수식**

$$\delta_{\text{prev}}(t) = 1 - \cos(s_{\{t-1\}}, s_t)$$
- **반환** — δ_{prev} (float). 첫 turn(직전 발화 없음)엔 None.
- **의미** — SEM 관점에서 “다음 발화도 직전과 같을 것”(identity dynamics)이라 가정한 사건모델의 예측 오차 그 자체.
- **한계** — 같은 주제 안에서도 잡담하며 말투·소재가 turn 마다 흔들려 δ_{prev} 가 된다 → 없는

경계를 만든다(과분할). 이 약점을 다음 함수가 보완한다.

함수 `_delta_ctx(s)` — 인과 윈도우 맥락 (긴 시간 척도)

hi_dots.py L145

- **역할** — 직전 한 발화 대신 **최근 m 개 발화의 평균 분위기**와 비교. 더 긴 시간 척도의 놀람.
- **수식**
$$c_{\{t-1\}} = \text{normalize}(\sum_{\{i=1..m\}} \rho^{\{i-1\}} \cdot s_{\{t-i\}})$$
$$\delta_{\text{ctx}}(t) = 1 - \cos(c_{\{t-1\}}, s_t)$$
- **파라미터**
 - `m = ctx_window = 2` — 과거 m 개 발화. **과거만 — 미래 안 봄 (causal, online)**.
 - `$\rho = \text{ctx\_decay} = 0.7$` — 기하감쇠. 직전 $\rho^0=1$, 그 전 0.7 ... 최근일수록 비중 ↑.
 - `normalize` — 길이 1 로 (cosine 은 방향만).
- **반환** — `$\delta_{\text{ctx}}$` (float). 과거 발화가 없으면 `None`.
- **의미** — 한 발화가 어쩌다 튀어도 최근 m 개를 섞은 `$c_{\{t-1\}}$` 은 그 흔들림이 평균에 묻혀 둔해진다 → 잡담성 팀에 덜 속는다. `_recent` 버퍼는 STEP 3 에서 이번 발화를 넣기 전 상태라 과거만 담는다(causal 보장).

함수 `_delta_eff(s)` — 두 척도 합성

hi_dots.py L161

- **역할** — 위 두 함수를 호출해 최종 놀람 신호를 만든다.
- **수식**
$$\delta_{\text{eff}}(t) = a \cdot \delta_{\text{prev}} + (1-a) \cdot \delta_{\text{ctx}} \quad (a = \text{ctx_blend_a} = 0.5)$$
- **반환** — `$\delta_{\text{eff}}$` (float). `$\delta_{\text{ctx}}$` 가 `None` 이면 `$\delta_{\text{eff}} = \delta_{\text{prev}}$` , `$\delta_{\text{prev}}$` 도 `None`(stream 맨 처음)이면 `$\delta_{\text{eff}} = 0$` .
- **의미** — `$\delta_{\text{prev}}$` (예민, 진짜 경계 안 놓침)와 `$\delta_{\text{ctx}}$` (둔감, 잡담에 안 속음)의 가중 평균. **이 합성이 Hi-DoTS 의 핵심** — 단일 척도가 아니라 두 시간 척도의 예측 오류를 함께 누적한다.

STEP 2 — 경계 판정 (assign)

hi_dots.py L190–197 — `assign` 이 STEP 1 의 `$\delta_{\text{eff}}$` 를 받아 경계를 판정한다.

첫 turn (topic_id 아직 없음) : is_boundary = False, segment 0 개시

그 외 : is_boundary = ($\delta_{\text{eff}} \geq \delta^*$)

- δ^* = 학습된 임계값 (delta_star, default 0.5594). “이 코퍼스·인코더에서 놀람이 이 정도를 넘으면 주제 변경”의 컷.
- 경계면($\delta_{\text{eff}} \geq \delta^*$) _topic_id 를 1 증가 → 새 segment 시작. 아니면 현재 segment 를 이어간다.
- _topic_id 는 단조 증가하는 segment 카운터 — 한 번 닫힌 segment 로 되돌아가지 않는다(online, 비가역).

이 한 줄 부등식이 Hi-DoTS 경계 결정의 전부다. (왜 prior·argmax·likelihood 같은 게 없어도 되는지는 § 4.)

δ^* 는 인코더·데이터셋·(m,ρ,a) 에 의존하므로 **held-out(train) split 에서 calibration 필수** — test 로 맞추면 leakage. 코드 default 0.5594 는 v4.1.x causal config (m=2,ρ=0.7,a=0.5) 의 TIAGE-train 산출값.

STEP 3 — graded score + 상태 갱신 (assign)

hi_dots.py L188, 199–216 — 같은 assign 호출의 마무리.

graded score:

$\text{graded_score} = \delta_{\text{eff}} / \delta^*$

STEP 2 의 결정 부등식이 $\delta_{\text{eff}} \geq \delta^*$ 이므로, $\text{graded} = \delta_{\text{eff}}/\delta^*$ 는 그 부등식을 1 을 기준으로 정규화한 연속값이다:

- $\text{graded} < 1 \rightarrow$ 경계 아님 (segment 이어가기)
- $\text{graded} \geq 1 \rightarrow$ 경계
- 1 에서 얼마나 멀리 떨어졌나 = **경계의 강도**

binary is_boundary 만으로는 “약한 경계”와 “확실한 경계”를 구분 못 한다. graded score 가 그 강도를 준다 (활용법 § 3). turn 1(δ_{eff} 정의 안 됨)은 $\text{graded} = 0$.

public 상태 기록: - last_delta_eff, last_graded_score, last_is_boundary 갱신 - _history 에 한 줄 append: {turn, topic_id, is_boundary, delta_eff, graded_score}

streaming 상태 갱신 (다음 turn 준비): - _prev_s = s (다음 turn 의 δ_{prev} 용) - _recent.append(s), 길이가 ctx_window(2) 넘으면 맨 앞 버림 (다음 turn 의 δ_{ctx} causal window 용)

반환: (topic_id, is_boundary).

복잡도

turn 당 연산은 $c_{\{t-1\}}$ 계산(과거 m 개 발화 가중합) 한 번뿐 → **turn 당 $O(m)$, $m=2$ 상수이므로 사실상 $O(1)$** . 누적 `state(_prev_s, _recent)`만 업데이트한다. 양측 발화를 비교해 depth score 를 계산하는 offline 방식이 turn t 결정에 누적 $O(t)$ 를 요구하는 것과 대비된다.

3. graded score 를 어떻게 쓰나

binary 경계만으로는 다운스트림이 약/강 경계를 구분 못 한다. **graded score** 는 그 강도를 연속값으로 준다. 신경과학적으로도 Ben-Yakov & Henson 2018 이 해마의 boundary 반응이 binary 가 아닌 **graded profile** 임을 보였고, `graded_score` 가 이와 직접 대응한다.

Boundary strength bands (`boundary_strength()` 가 이 구간으로 히스토그램, [hi_dots.py:233](#)):

band	graded_score	downstream 권고
very_weak	< 0.7	보류 (decision deferred)
weak	$0.7 \sim 1.0$	segment 내부 (boundary off)
normal	$1.0 \sim 1.3$	정상 경계
strong	≥ 1.3	즉시 commit

Per-band precision (그 band 의 turn 중 실제 GT 경계 비율, `outputs/experiments/2026-05-21_v413_demo` — Hi-DoTS 출력이 v4.1.3 와 동일하므로 그대로 유효):

dataset	very_weak	weak	normal	strong (≥ 1.3)
TIAGE	0.000	0.238	0.345	0.520
Dialseg711	0.000	0.129	0.383	0.800

→ **graded score** 가 진짜 calibrated 신호다: `very_weak` 는 거의 경계가 아니고, `strong band` 는 precision 0.52~0.80. SeCom 같은 uncertainty-aware 다운스트림 consumer 가 “strong → 즉시 메모리 commit, `very_weak` → 보류” 식으로 직접 활용할 수 있다.

4. 왜 이게 전부인가 — v4.1.x dead-code 감사 요약

“prior 도, likelihood 도, Bayes argmax 도 없이 부등식 하나로 되나?” — 된다. Hi-EM v4.1.x (HiEMSegmenterV411/V413) 는 SEM2 기계를 다 갖고 있었지만, 2026-05-22 감사에서 v4.1.3 default

config 기준으로 그 전부가 **출력에 무의미함**이 증명됐다:

- 학습형 **per-topic RNN** — $\eta_{\text{prev}}=1$ 이라 RNN 항의 가중치가 정확히 0. forward 도, 학습도 안 일어남. (RNN 객체는 lazy 생성이라 만들어지지도 않음.)
- **f0 / restart / 재진입 분기** — $f0_{\text{min_starts}} \geq 2$ 에서 순환 교착: 매 boundary 가 새 topic_id 를 만드니 한 topic 이 episode-start 를 2번 꺾을 일이 없어 f0 가 영영 untrained → 분기가 한 번도 발화 안 함.
- **SEM2 분산 기계** (per-topic σ_k^2 , scaled-inverse- χ^2 posterior) — 죽은 f0 likelihood 에만 입력될 뿐 어디에도 안 닿음.
- **sticky-CRP prior** (α/λ) + 고정 **softness 온도** $\sigma\delta^2$ — $_scores$ 가 fresh slot 에 prior-corrected baseline $B0_t = L\delta(\delta^*) + \log(\text{prior 비율})$ 를 쓰도록 설계돼 있어, repeat-vs-fresh argmax 에서 prior 항과 $\sigma\delta^2$ 가 **정확히 상쇄**된다.

→ 그래서 Bayesian posterior argmax 가 **정확히·증명 가능하게 $\delta_{\text{eff}} \geq \delta^*$ 단일 임계값 판정으로 환원**된다. 경험적으로도 위 하이퍼파라미터들은 각자 전 범위에서 **byte-identical 한 segmentation** 출력을 낸다 — 즉 그것들은 조정해도 결과가 안 바뀌는, 출력상 존재하지 않는 변수다.

Hi-DoTS 는 그 환원형이다. SEM2 기계는 틀려서 버린 게 아니라 이 도메인·이 설정에서 증명 가능하게 degenerate 해서 main 모델에서 제외하고 archive/legacy_sem_ablation/ 에 실행 가능한 감사 계보로 보존한다.

Hi-DoTS 와 SEM 의 관계: Hi-DoTS 는 SEM 의 핵심 직관 — “다음 관측이 최근 사건 맥락으로 잘 예측되지 않을 때 사건 경계가 생긴다” — 의 최소 온라인 구현이지, SEM2 의 충실한 포팅이 아니다.

5. HP default

```
delta_star = 0.5594 # 경계 임계값  $\delta^*$  (인코더·데이터셋별 calibration 필수)
ctx_window = 2      # m: causal 맥락에 쓰는 과거 발화 수
ctx_decay = 0.7     #  $\rho$ : 기하감쇠 (최근일수록 가중  $\uparrow$ )
ctx_blend_a = 0.5   # a:  $\delta_{\text{eff}} = a \cdot \delta_{\text{prev}} + (1-a) \cdot \delta_{\text{ctx}}$ 
```

생성자는 dim(임베딩 차원, API parity·검증용) 도 받는다. δ^* 만 데이터 의존이고 나머지 셋은 (m, ρ , a) calibration 산출값이다. 인코더 = 우리 기본 multi-qa-mpnet.

ctx_window default = 2 인 이유: § 6 의 보고 수치(0.4675 / 0.5897 / 0.4631)를 낸 canonical TIAGE-cfg 가 m=2 다. v4.1.x 코드 default 는 ctx_window=3 이었으나 보고 config 와 어긋났던 것으로, Hi-DoTS 는 main 모델로서 default 를 보고 config 에 맞춰 2 로 둔다 (2026-05-22 정정).

6. 결과 (공식 SuperDialseg metric, $\text{Score}=0.5F1+0.25(1-Pk)+0.25(1-WD)$)

동일 config(m=2)에서 Hi-DoTS 출력이 v4.1.3 와 **byte-identical** 임이 검증됐다 — 세 벤치 38,242 turn 중 boundary 예측이 다른 turn 0개, Score 까지 자리수 동일 (scripts/verify_hidots_parity.py, outputs/experiments/2026-05-22_hidots_parity/REPORT.md). 따라서 수치는 v4.1.3 와 같다:

벤치(test)	Score	F1	Pk	WD
TIAGE △	0.4675	0.410	0.442	0.508
Dialseg711	0.5897	0.549	0.325	0.415
SuperDialseg	0.4631	0.432	0.471	0.541

△ TIAGE-test 는 calibration source(TIAGE-train)와 같은 corpus → in-domain 측정. Dialseg711·SuperDialseg 는 zero-shot transfer. 세 벤치 모두 동일 인코더 위 단일-임계값 prev-cos 기준선 대비 일관 우위(특히 Dialseg711 zero-shot).

7. 한계 / 검증 미해결

- δ^* 가 TIAGE-train 전이값 — SuperDialseg validation 재calibration 미완. F1-best δ^* 가 곧 Score-best δ^* 가 아님이 확인됨(F1-best 는 과분절 유발) → 향후 Score/WD-aware calibration 필요.
- graded score band 임계값(0.7 / 1.0 / 1.3)은 휴리스틱 — Ben-Yakov & Henson 실험 데이터에서 직접 fit 한 값이 아니다. downstream consumer 실험으로 보정 필요.
- very_weak band(<0.7)에 stream-start turn(graded=0)이 섞인다. boundary 통계 낼 때 caller 가 turn 0 을 걸러야 한다.
- 인접-cosine unsupervised 천장이 벤치별 상이 (SuperDialseg $\approx$ F1 0.46, Dialseg711 $\approx$ 0.54). 문헌 SOTA(0.55~0.65)는 출처·split·supervised·인코더 미확정 → 외부 초과주장 금지. 정합 인코더 ablation 미실행.
- Hi-DoTS 는 wording 은 비슷하나 topic 이 바뀌는 경우($\delta \approx$ 낮음인데 GT 경계) 를 못 잡는다 — 인접 의미거리 신호 자체의 구조적 한계. 예상 현실적 F1 천장 $\approx$ 0.48~0.55.

검증 노트 (2026-05-22, 코드 1:1 대조)

- § 2 동작 순서는 HiDoTS.assign (hi_dots.py:180) 의 실제 호출 순서와 1:1 대조 — _delta_eff → 경계 판정 → graded + 상태 갱신, 그 외 호출 없음.

- `_delta_eff` 의 두 fallback 확인: $\delta_{\text{prev}}=\text{None}(\text{stream 시작})\rightarrow 0.0$, $\delta_{\text{ctx}}=\text{None}\rightarrow\delta_{\text{prev}}$.
- § 4 의 “v4.1.x $\rightarrow$ Hi-DoTS 환원” 주장은 별도 dead-code 감사 (archive/legacy_sem_ablation/sem_core_v413.1) 대조 + HP byte-identity 실험)의 결론이며, Hi-DoTS docstring 의 “output parity” 주장과 일치.
- $\text{graded score} = \delta_{\text{eff}}/\delta^*$, turn 0 = 0.0, band 경계(0.7/1.0/1.3) 코드 ([hi_dots.py:233](#))와 일치.